ロボットによる6面外観検査ライン
止まらないレーンで、カメラとアームを同期させる

Physical AI 応用編1 最終課題　説明資料　／　MuJoCo シミュレーション ＋ ROS 2 ＋ 画像認識AI（ResNet-18）

一言でいうと：止まらずに流れてくる切削部品を、カメラの映像から AI が位置と向きを推定し、ロボットアームがベルトと同期して追いかけてつかみます。 つかんだ部品は手首の回転と持ち替えで6面すべてをカメラに見せ、AI が面ごとに判定します。不良品は NG 箱へ、良品は流れているレーンの元の位置へ戻します。 推定がアームに届くまでの遅れを撮影時刻で補正することで、遅れが 1.5 秒あってもつかみ成功率 100% を保ちました（補正なしは 0%）。 6面検査の不良品検出率は 100%、上面だけの検査では 11% でした。

1. 課題設定：どんな現場の、何を自動化するか

想定する現場	精密切削工場の出荷前工程。切削・洗浄を終えた部品を、梱包前に全数検査する（本課題で設定した想定）。
対象部品	アルミ（A6061）削り出しのセンサーハウジング。40 mm 角、上面に取付ねじ穴 1 か所。
見つけたい不良	打痕・傷（搬送や治具との接触）、油ジミ（洗浄残り）、角欠け。6面のどこにでも出る。
流れ方	前工程から、コンベアで1個ずつ流れてくる。間隔はばらばらで、ガイドの範囲（±15 mm）で横にずれ、向きもそろっていない。
ねらい	レーンを止めずに、全6面を毎回同じ条件で撮って判定する。良品は元の並び順のまま次工程へ流す。

このタスクには、ロボットの動きの面で難しい点が3つあります。

1. **部品は止まってくれない。**位置も向きも毎回違う部品を、動いているベルトの上でつかむ必要がある。
2. **つかむと見えない面ができる。**指が触れている2面は、つかんでいる間は撮れない。
3. **カメラとアームの間には遅れがある。**画像を撮ってから AI が推定し、アームに届くまでの間にも部品は進む。

2. ラインの設計

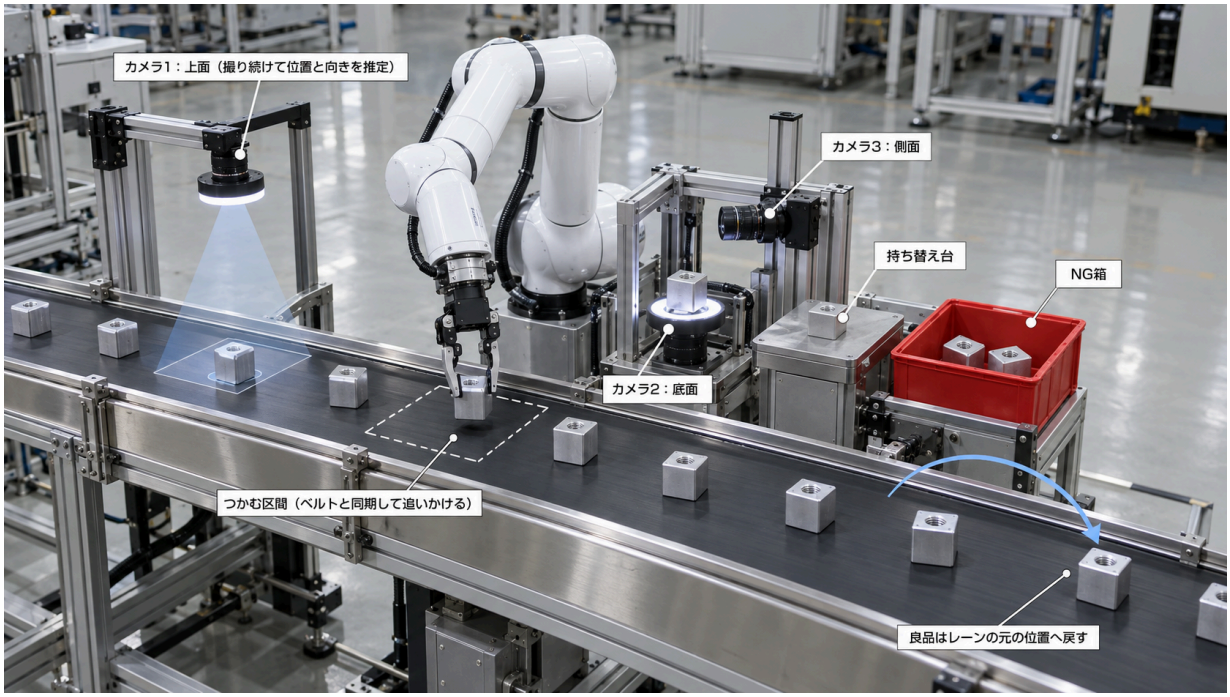


図1　ラインの構想図（生成AIで作成したイメージ）。実装はこの配置を MuJoCo 上に再現した。

1個あたりの流れ

1. **カメラ1（上面）が撮り続ける。**1秒に10枚撮り、部品が写っている間、AI が毎フレーム位置と向きを推定する。いちばん中央に写ったフレームで上面を検査する。
2. **追いかけてつかむ。**部品が「つかむ区間」に入ったら、アームがベルトと同じ速さで追いかけながら降りてつかむ。レーンは止めない。
3. **検査台で底面と側面。**カメラ2（下カメラ＋リング照明）が底面を、カメラ3（横カメラ1台）が手前の側面を撮る。手首（第7関節）だけを180°回して反対側の側面も撮る。
4. **持ち替え。**持ち替え台に置き、90°回してつかみ直す。最初に指で隠れていた2面を、カメラ3と手首の回転で撮る。
5. **仕分け。**どこか1面でも不良なら NG 箱へ。6面とも良なら、流れているレーンの元の位置へ追いかけて戻す。不良が見つかった時点で残りの撮影を打ち切る。

カメラは上面・底面・側面の3台だけです。側面4面は、手首の回転と持ち替えで1台のカメラに向けます。指で隠れた面はカメラを増やしても撮れないため、持ち替えは必ず必要になります。 公開されている多面検査の製品にも、ロボットで向きを変えて少数のカメラで撮る方式があります（スカイロジック「検査員ロボット」、ヒューブレイン「多面外観検査装置」など）。

3. 動作と制御の工夫

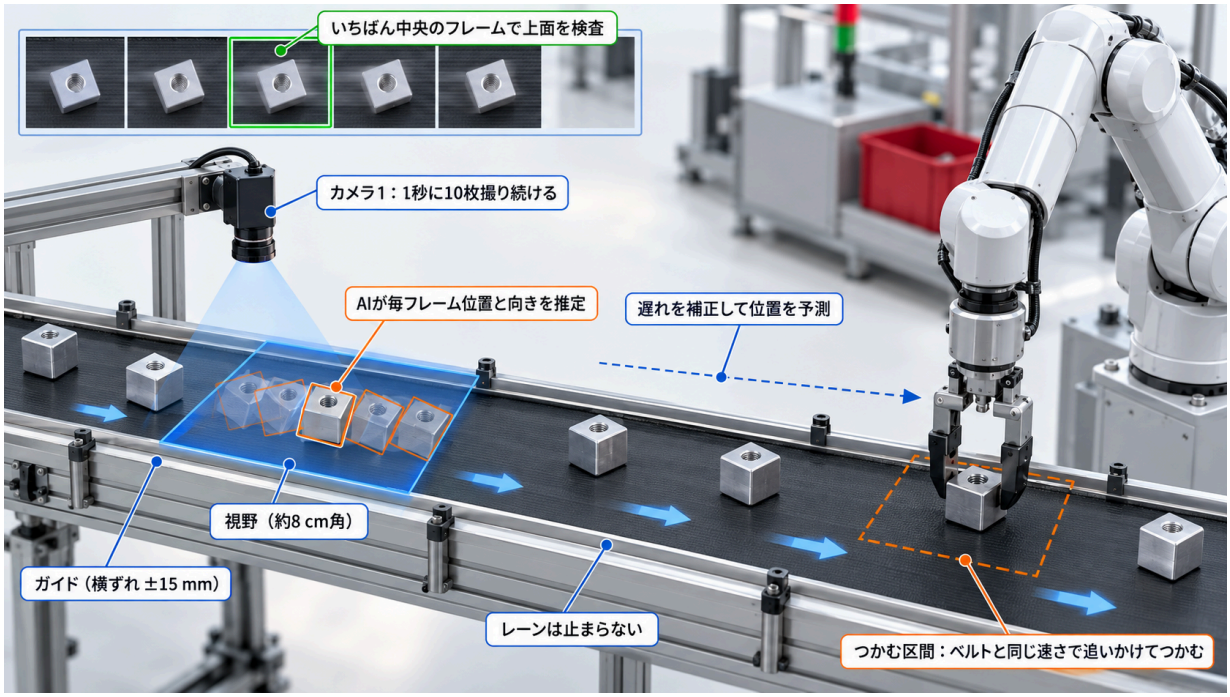


図2　カメラ1の撮影から、追いかけてつかむまでの流れ（生成AIで作成したイメージ）

3.1 カメラとアームの同期（止まらないレーンでつかむ）

1. **撮り続けて、写ったことを知る。**カメラ1は部品が真下に来るのを待たずに撮り続ける。画像の明るい部分の割合で、部品が視野に入ったことと出たことを判定する（光電センサーの代わり）。
2. **毎フレーム推定する。**AI（ResNet-18）が、部品の横・前後の位置と、向き（90°ごとに同じ形なので −45〜45° の範囲で表す）を推定する。
3. **撮影時刻と回転計を組み合わせる。**各フレームには、撮影した瞬間のベルトの回転計（エンコーダ）の値を記録する。「ベルトの上のどこに部品が載っているか」は、部品全体が写ったフレームの推定から求める。そのあとの部品の位置は回転計だけで追う（工場のコンベア追従と同じ考え方）。
4. **遅れを補正する。**推定がアームに届くのは、撮影から推論と通信の時間だけ後になる。届いた時刻の回転計の値と組み合わせると、その間に進んだ分だけ後ろを狙ってしまう。撮影時刻の値と組み合わせることで、この遅れを打ち消す。
5. **追いかけて、つかめたかを確かめる。**予測位置へ手先を合わせ、ベルトと同じ速さで追いかけながら降りて指を閉じる。指の開きが部品の幅で止まり、部品が指の間にあるときだけ「つかめた」とする。外れたら記録し、その部品は未検査として扱う。

最初はカメラの映像からベルトの速さも推定していましたが、約5%の誤差が、つかむまでの十数秒で1〜2 cm のずれに積み上がり、つかみ損ねが多発しました。回転計と組み合わせる方式に変えて解決しました。

3.2 つかむと隠れる面を見せる動き

- **手首だけを180°回す。**腕全体で部品を裏返すと腕が大きく振れて部品を落としたため、第7関節だけを回して反対側の面を同じカメラへ向ける。
- **関節の限界を見て、つかむ向きを選ぶ。**同じ面をつかむ向きは2通り（180°違い）ある。逆運動学で両方を試し、第7関節が限界から離れていて、さらに180°回す余裕が残る向きを選ぶ。横カメラも、第7関節が可動範囲の中央付近になる位置（検査台の −x 側）に置いた。
- **持ち替えで90°変える。**持ち替え台に置いてから、手先を90°回してつかみ直し、最初に指が触れていた2面を露出させる。

- **動くレーンの元の位置へ戻す。**つかんだ時点のベルト上の位置を覚えておき、回転計で今どこにあるかを計算して、そこへ追いかけて置く。

3.3 逆運動学と物理の扱い

- 手先の目標位置・向きから7つの関節角を求める逆運動学を自作した（減衰最小二乗法。余った自由度で基準姿勢に寄せる）。
- 指が部品に触れている間は、ベルトが部品の下を滑り、部品は手と一緒に動く。
- つかんだ後は、部品を手先に固定する拘束を使う（把持の補助。8章の限界に記載）。

4. AI の役割

AI	入力 → 出力	学習データと精度
① 位置・向きの推定（動きを決める）	カメラ1の画像 → 部品の横・前後の位置、向き。アームの目標位置とつかむ向きになる。	位置・向き・照明・ベルトの縞の位置をばらつかせた合成画像 8,000 枚。誤差は横 0.7 mm、前後 1.0 mm、向き 0.6°。1枚あたり約 56 ms（CPU）。
② 面ごとの良否判定（行き先を決める）	各カメラの面画像 → 良・打痕傷・油ジミ・角欠け。1面でも不良確率が0.5を超えたら NG。	ラインと同じカメラ・姿勢で、照明・色味・位置・向きをばらつかせた合成画像 16,000 枚。テスト 1332 枚で正解率 100.0%。

合成画像の不良ははっきり写るため、判定の精度は高く出ます。実際の金属の反射や限度ぎりぎりの小さな傷は、8章の限界に記載しました。

5. 評価

5.1 遅れの補正と、つかみ成功率

推定がアームに届くまでの遅れを 0〜1.5 秒で変え、補正あり・なしで、流れる部品をつかめた割合を比べました（各条件 24 個）。

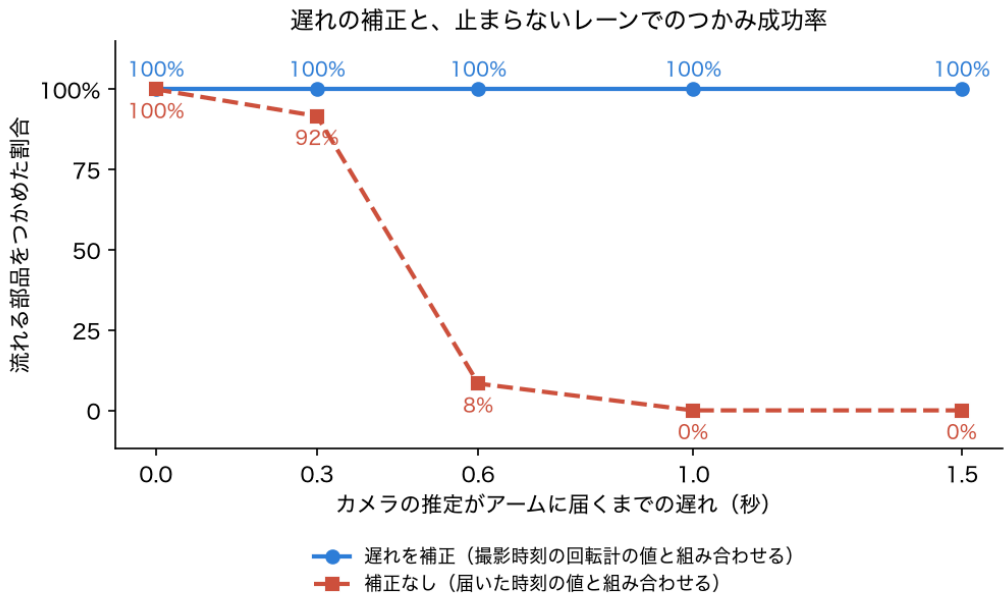


図3　遅れとつかみ成功率

遅れ（秒）	0.0	0.3	0.6	1.0	1.5
補正あり	100%	100%	100%	100%	100%
補正なし	100%	92%	8%	0%	0%

補正なしでは、遅れの間に進んだ分（ベルト速度 3 cm/秒 × 遅れ）だけ後ろを狙うため、遅れが大きいほどつかみ損ねが増えます。撮影時刻で補正すると、遅れが大きくても成功率を保てました。

5.2 検査方式の比較

同じ止まらないレーン・同じ部品で、検査方式だけを変えて流しました（6 ロット、計 48 個、うち不良 18 個）。

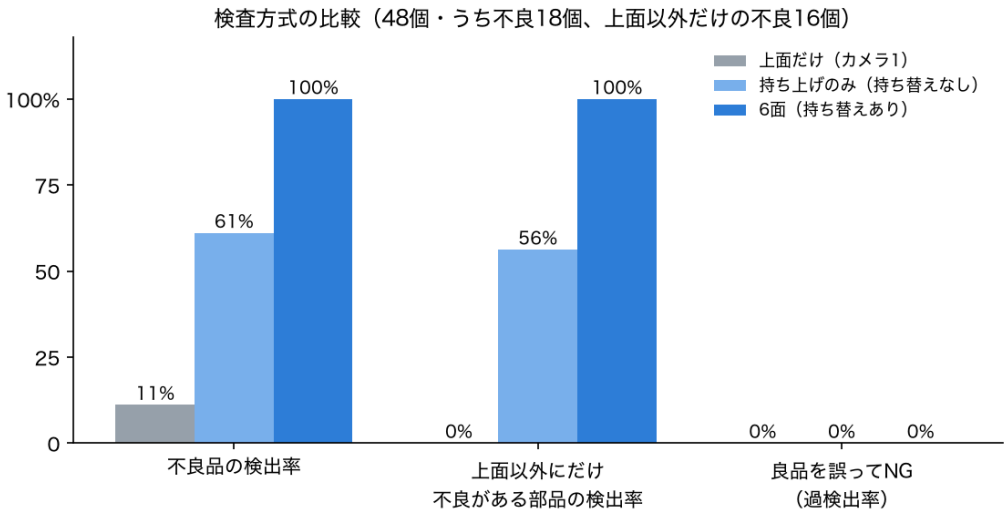


図4　検査方式ごとの不良品検出率と過検出率

方式	不良品の検出率	上面以外にだけ不良がある部品	良品の誤NG	つかみ成功率	つかんだ1個あたりのアーム作業（不良品を含む平均）
上面だけ（カメラ1）	11%	0%	0%	100%（2/2）	—
持ち上げのみ（持ち替えなし）	61%	56%	0%	100%（48/48）	9.3 秒
6面（持ち替えあり）	**100%**	**100%**	0%	100%（48/48）	15.9 秒

上面だけでは、上面以外の不良はすべて見逃します。持ち上げるだけでは、指で隠れた2面の不良を見逃します。持ち替えまで行うと、6面すべてを確認できます。

つかみ成功率のかっこ内は「つかめた回数／つかもうとした回数」です。上面だけの方式は、上面で不良が見つかった部品だけをつかんで NG 箱へ運ぶため、回数が少なくなります。

5.3 処理時間と撮影時刻の確認

第3回の演習にならい、AI の処理が1フレームの時間に収まるかを測りました。1ロットで実際に使った画像（カメラ1 272 枚、面画像 43 枚）を、ウォームアップの後に3回ずつ順番を混ぜて推論し、中央値と p95 を求めました。描画と結果の照合は計測から外しています。

AI	中央値	p95	予算
① 位置・向きの推定（カメラ1の毎フレーム）	8.6 ms	9.5 ms	100 ms（カメラ1の撮影間隔、10 Hz）
② 面ごとの良否判定（検査の1枚ごと）	8.5 ms	9.0 ms	—

撮影時刻の間隔はすべて 100 ms で、抜けたフレームは 0 枚でした。各フレームの回転計の進みはベルト速度×間隔と一致し、保存した画像は描画結果のコピーで、同じメモリを共有していないことも確かめました。数値はこのPCのCPUでの値です。5.2 の評価（7プロセスを並列・各1スレッド）では、位置推定は平均 56 ms でした。どちらも撮影間隔に収まり、遅れの実験で与えた 0.3 秒は、これに通信の時間を見込んだ値です。

6. ROS 2 での構成

撮影するライン側と判定AIを、ROS 2 の2つのノードに分けて動かせるようにしました。判定AIを GPU のある別のPCに置くなどの構成変更に対応できます。

ノード	やり取り
conveyor_line（MuJoCo＋ロボット）	面画像を `/inspection/image` へ送り、`/inspection/verdict` の判定を待つ。関節の状態 `/joint_states`、作業の状態 `/line/status`、部品ごとの結果 `/line/result` を配信する。
inspector（判定AI）	`/inspection/image` を受けて判定し、`/inspection/verdict` に返す。

7. ベースにしたサンプルと、発展させた点

課題資料のサンプル「Pick and Place Task（UR3＋2指グリッパ、ROS 2＋Gazebo）」（資料の難易度：中級）を出発点にしました。サンプルの内容は、動画の説明欄と公開コード（IFRA-Cranfield の ros2_SimRealRobotControl と IFRA_LinkAttacher）の手順書で確かめました。サンプルは、YAML に書いた決まった動作（関節角への移動・直線移動・グリッパの開閉）を順に実行し、つかんだ物はプラグインで手先と固定ジョイントでつなぎます。

サンプル	本課題
止まっている物を、決まった場所でつかむ	止まらないレーンを流れる、位置も向きもばらばらな部品を、**カメラと同期して追いかけてつかむ**
カメラとアームの遅れは扱わない	推論と通信の遅れを**撮影時刻と回転計で補正**し、効果を成功率で示す
つかんで運ぶことが目的	つかむと隠れる面を、**持ち替えと手首の回転**で見せる。関節の限界を見てつかむ向きを選ぶ
置き場所は固定	動いているレーンの**元の位置へ追いかけて戻す**
つかむ指令で、物を手先に固定ジョイントでつなぐ	同じく手先に固定するが、その前に**指の開きと部品の位置で、挟めたかを確かめる。**挟めなければ固定せず、つかみ損ねとして記録する
AI なし	AI が位置と向きを推定して動きを決め、面ごとに良否を判定する

ほかのサンプルは、資料の難易度で次のように分かれています。初級は ROS の入門（差動二輪ロボットを速度指令で走らせる、Gazebo との連携）です。中級には、人のお手本から模倣学習（ACT）で動作を学ぶ講義資料や、VLM（Gemini Robotics-ER）で物体の位置や動作の手順を得る例があります。中〜上級は強化学習（Genesis）で4脚の歩行を学ぶ例、上級は四足ロボットの製作・歩行デモと、ChatGPT API による自

然言語での操作です。本課題は、AI を自分で学習させて「見る」側に使い、その推定とカメラ・回転計の同期で動作を決める方向に発展させました。

8. 限界

- **把持の補助：**つかんだ後は、部品を手先に固定する拘束で運びます。接触の摩擦だけだと、シミュレーション上で部品が指の間で滑ったためです。つかめたかどうかの判定は、指の開きと部品の位置で別に行っています。
- **シミュレーションの画像：**学習も評価も合成画像です。実機では、金属面の反射、照明、限度ぎりぎりの小さな傷への対応が課題になります。
- **評価の数：**不良品の検出率は、不良 18 個での結果です。見逃しゼロを保証するには、より多くの不良品で確かめる必要があります。
- **遅れは設定値：**5.1 の遅れは、シミュレーション上で与えた一定の値です。ROS 2 の通信遅延の実測や、遅れの揺らぎは含めていません。
- **動きは手順として設計：**ロボットの動作は逆運動学と手順で作り、学習による動作生成は使っていません。AI は位置・向きの推定と良否判定に使っています。次の段階として、この動作をお手本のデータにして、模倣学習（第5回の ACT など）で動作そのものを学習させることを考えています。

9. 使用したもの

項目	内容
主なソフトウェア	MuJoCo、ROS 2 Humble（RoboStack）、PyTorch / torchvision（ResNet-18）、NumPy、imageio、Pillow、matplotlib
ロボットモデル	MuJoCo Menagerie の Franka Emika Panda（Apache-2.0）。リポジトリには含めず、スクリプトで取得。
コード	MIT License。再現手順はリポジトリの README に記載。
生成AIの利用	構想図（図1・図2）の作成と先行事例の調査に生成AIを使用した。コードの作成・デバッグにもAIアシスタントを使った。
動画	デモ部分は2倍速（動画内に表示）。